Module 4
Iteration Control Blocks

Agenda

1. Introduction to Loops
2. Types of Loops
3. Definite Looping: for Loop
4. Generating Sequences with range()
5. Indefinite Looping: while Loop
6. Infinite Loops & Unsafe Looping
7. Loop Interruption
8. State Tracing Strategies

© Copyright Intellipaat. All rights reserved.

Introduction to Loops

© Copyright Intellipaat. All rights reserved.

Repetition Around Us

➤ **Think about your daily life.**

Do you perform these activities repeatedly?

- Waking up every morning
- Attending classes every day
- Practicing sports regularly
- Brushing your teeth daily



What is Repetition?

Repetition means performing the same task again and again.

Question for You

If a task repeats many times, should we keep doing it manually every time?

© Copyright Intellipaat. All rights reserved.

Repetition in Programming

- In programs, we often need to perform the same task many times.

Example

Suppose we want to print "Hello" 5 times.

```
1  print("Hello")
2  print("Hello")
3  print("Hello")
4  print("Hello")
5  print("Hello")
```

Output

```
Hello
Hello
Hello
Hello
Hello
```

Problems with Manual Repetition

- Too much code
- Time-consuming
- More chances of mistakes
- Difficult to maintain

Think!

What if we want to print "Hello" 100 times?
Should we write 100 lines of code like this?



Manually writing the same code again and again is not a good idea!

Key Takeaway

Repetition is common in programming, but writing the same code again and again is inefficient.
We need a better way to handle repetition.

© Copyright Intellipaat. All rights reserved.

Why Do We Need Loops?

Problems with Manual Repetition

Too much code
The program becomes long and hard to read.

Time-consuming
Writing many lines of code takes more time.

More chances of mistakes
Copy-pasting code can lead to errors.

Difficult to maintain
If we need to change something, we have to do it in many places.



Loops are the solution!

Loops allow us to write a block of code once and execute it automatically based on a condition or number of times.

With loops, we can:

- Reduce number of lines of code
- Save time and effort
- Improve code readability and maintainability
- Make our programs more efficient and reliable

```
for i in range(10):
    print("Welcome to Python")
```

↓

Same output with just 2 lines of code!

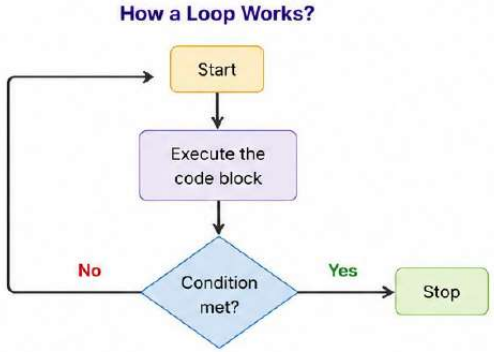
© Copyright Intellipaat. All rights reserved.

What is a Loop?

A loop is a programming structure that allows a block of code to execute **repeatedly** until a condition is met.

Key Idea

When repetition exists in a program, we use **loops** to automate it.



Simple Loop Example

Without Loop (Manual Repetition)

```
print("Hello")
print("Hello")
print("Hello")
print("Hello")
print("Hello")
```

- More lines of code
- Time-consuming
- Hard to maintain
- More chances of mistakes

VS

With Loop (Using for Loop)

```
for i in range(5):
    print("Hello")
```

Output:

```
Hello
Hello
Hello
Hello
Hello
```

- Less code
- Faster to write
- Easy to maintain
- Fewer chances of mistakes

Key Takeaway: Loops help us write less code and perform repetitive tasks efficiently.

© Copyright Intellipaat. All rights reserved.

What is Iteration?

An iteration is **one complete execution (one cycle) of a loop.**

When a loop repeats a block of code, each repetition is called an **iteration.**

➢ Loop = **Many Iterations**

Think of it like this...

A Loop is like a machine → **It repeats** a task → **Each repeat** is one **iteration**

Example

Look at the simple Python code:

```
for i in range(3):
    print("Hello")
```

Output

```
Hello
Hello
Hello
```

What's happening here?

1. 1st execution → **Iteration 1**: prints "Hello"
2. 2nd execution → **Iteration 2**: prints "Hello"
3. 3rd execution → **Iteration 3**: prints "Hello"



Quick Connection

Repetition — Doing something again and again → **Loop** — The structure that performs repetition → **Iteration** — One cycle of loop execution

Key Takeaway: Iteration is the heart of every loop. Understanding iteration helps us understand how loops actually work!

© Copyright Intellipaat. All rights reserved.

Hands-On: Introduction to Loops

© Copyright Intellipaat. All rights reserved.

Hands-On: Introduction to Loops

Problem Statement

A **fitness application** must display a motivational message to **encourage users to exercise.**

Instead of writing the same `print()` statement repeatedly, use a **loop** to to display the message exactly **5** times.



Expected Output

```
Keep moving and stay fit!
Keep moving and stay fit!
Keep moving and stay fit!
Keep moving and stay fit!
Keep moving and stay fit!
```



© Copyright Intellipaat. All rights reserved.

Types of Loops

© Copyright Intellipaat. All rights reserved.

Are All Loops the Same?

We already know that a loop helps us repeat a block of code.

But do all loops repeat in the same way?



Think About It

Consider the two situations below.
Do both tasks repeat in the same way?

1 Situation 1

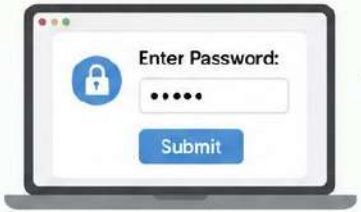
Print "Hello" 5 times.

```
print("Hello")
print("Hello")
print("Hello")
print("Hello")
print("Hello")
```

→ Repetitions are known (5 times)

2 Situation 2

Keep asking for password until the correct password is entered.



→ Repetitions are unknown (until correct)

© Copyright Intellipaat. All rights reserved.

Definite vs Indefinite Iteration

We classify loops based on whether the number of repetitions is known in advance or not.

1. Definite Iteration

Number of repetitions is **known beforehand.**

Examples

- Print numbers from 1 to 10
- Display 5 student names
- Run a loop 20 times
- Calculate sum of first N numbers

Example in Python

```
for i in range(5):
    print(i)
```

Output:

```
0 1 2 3 4
```

VS

2. Indefinite Iteration

Number of repetitions is **not known in advance.**

Examples

- Keep asking for PIN until correct
- Retry login until successful
- Read data until end of file
- Game runs until player quits

Example in Python

```
password = ""
while password != "1234":
    password = input("Enter PIN: ")
print("Access Granted!")
```

Output:

(Repeats until correct PIN entered)

Key Differences

Feature	Definite Iteration	Indefinite Iteration
Repetitions known?	Yes, we know in advance.	No, we don't know in advance.
Stop Condition	Based on count (e.g., 5 times, 10 times).	Based on a condition (e.g., correct input, end of file).
Common Loop Used	`for` loop	`while` loop
Control	More structured and predictable.	More flexible and condition-dependent.

Remember: **If we know** how many times to repeat → **Definite Iteration** (usually `for` loop)
If we don't know how many times → **Indefinite Iteration** (usually `while` loop)

© Copyright Intellipaat. All rights reserved.

Types of Loops in Python

for Loop

- Used when the number of iterations is known
- Iterates over a sequence (list, string, range, tuple, etc.)
- Easier to read and write
- Best for fixed repetitions

VS

while Loop

- Used when the number of iterations is unknown
- Runs until a condition becomes False
- Gives more control over execution
- Best for condition-based repetition

for Loop Syntax

```
for variable in iterable:
    # code to execute
```

Example

```
for i in range(5):
    print(i)
```

Output

```
0
1
2
3
4
```

while Loop Syntax

```
while condition:
    # code to execute
```

Example

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Output

```
0
1
2
3
4
```

© Copyright Intellipaat. All rights reserved.

Hands-On: Types of Loops

© Copyright Intellipaat. All rights reserved.

Hands-On: Types of Loops

Problem Statement

A warehouse has **6 storage sections** that need to be inspected every morning.

- Write a program that uses **one loop** to announce the inspection of each section.
- The program should **automatically update the section number** during every repetition instead of using six separate `print()` statements.



Expected Output

```
Inspecting Storage Section 1.
Inspecting Storage Section 2.
Inspecting Storage Section 3.
Inspecting Storage Section 4.
Inspecting Storage Section 5.
Inspecting Storage Section 6.
```



© Copyright Intellipaat. All rights reserved.

Definite Looping: for Loop

© Copyright Intellipaat. All rights reserved.

Why Do We Need a *for* Loop?

Sometimes, we need to repeat a set of instructions multiple times.
When we know the number of repetitions in advance, we use a `for` loop.

Situation

Print numbers from 1 to 5.

Without using a loop:

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

Question

Is writing every statement manually efficient?

Answer: No

Therefore,

Since we know the repetition count beforehand, we should use a `for` loop.

Key Idea

Use a `for` loop when the number of repetitions is known.

Examples

- Print numbers from 1 to 10
- Display 5 student names
- Run a task 20 times

© Copyright Intellipaat. All rights reserved.

Syntax of for Loop

The `for` loop in Python is written using the following syntax.

Syntax

Keyword — Loop Variable — Range of values (start, stop, step)

```
for variable in range(start, stop, step):
    statements
```

Block of code that will be executed in each iteration

Example

```
for i in range(1, 4):
    print(i)
```

Output

```
1
2
3
```

`range(1, 4)` generates values 1, 2, 3. Note that 4 is not included.

Explanation

`for`	Starts the **for** loop.
`variable`	Takes each value from the **range** one by one.
`in`	Keyword that connects the variable with the **range**.
`range(start, stop, step)`	Defines the range of values to iterate. • `start` → Starting value (included) • `stop` → Stopping value (excluded) • `step` → Step size (optional, default is 1)
`statements`	Set of instructions that are executed in every iteration.

Before moving ahead, let's understand range() in detail

© Copyright Intellipaat. All rights reserved.

Hands-On: Definite Looping — for Loop

© Copyright Intellipaat. All rights reserved.

Hands-On: Definite Looping — for Loop

Problem Statement

A grocery store has the following items:
Rice, Milk, Bread, and **Eggs.**

Use a **for** loop to visit each item
in the sequence one by one and print
a message showing that the item is in stock.



Expected Output

```
Rice is in stock.
Milk is in stock.
Bread is in stock.
Eggs is in stock.
```



© Copyright Intellipaat. All rights reserved.

Working with range() in *for* Loop

© Copyright Intellipaat. All rights reserved.

Why Do We Need range()?

When working with loops, we often need to repeat a block of code a **specific number of times**.

Situation

We want to print numbers from 1 to 100.

Using a list:

```
for i in [1, 2, 3, 4, 5, ..., 100]:
    print(i)
```

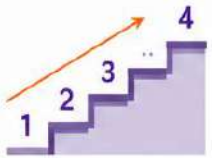
Question:

Should we manually write all 100 numbers? ✕ No!

Python provides **range()** to generate sequences of numbers **automatically**.

Real-Life Analogy

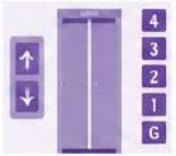
Think of **range()** like:



Counting Stairs

1, 2, 3, 4, 5, ...

You don't list every step, you just start and keep going.



Floor Numbers in a Building

Each floor number follows a pattern.



Jersey Numbers

Players are numbered in order without writing them one by one.

range() helps us generate such sequences **easily** and **efficiently**.

© Copyright Intellipaat. All rights reserved.

What is range()?

range() is a built-in Python function used to generate a **sequence of numbers**.

Syntax

```
range(start, stop, step)
```

- **start** → Starting point of the sequence (inclusive).
- **stop** → Ending point of the sequence (**exclusive**).
- **step** → Difference between two consecutive numbers. Default is 1.

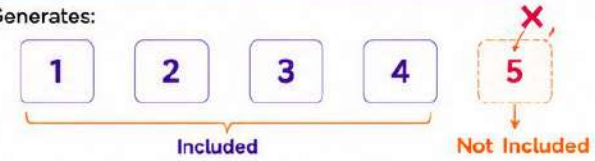
Important Note

The **stop** value is **exclusive**.
It is not included in the sequence.

Example

```
range(1, 5)
```

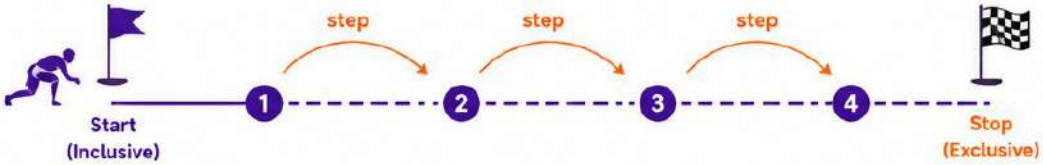
Generates:



Real-Life Analogy

Think of **range()** like a race track.

- You start at a point (**start**).
- You run until just before the finish line (**stop**).
- You move in fixed steps (**step**).



range() helps us create number sequences easily **without writing each number manually.**

© Copyright Intellipaat. All rights reserved.

Different Forms of range()

IntelliPaat

range() can be used in **three** different ways.

1 Form 1: range(stop)

Generates numbers from **0** to **stop-1**.

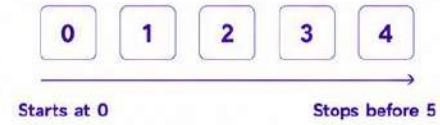
Syntax

```
range(stop)
```

Example

```
range(5)
```

Output



Useful when you want to **start from 0** by default.

2 Form 2: range(start, stop)

Generates numbers from **start** to **stop-1**.

Syntax

```
range(start, stop)
```

Example

```
range(2, 6)
```

Output



Useful when you want to start from a **specific number**.

3 Form 3: range(start, stop, step)

Generates numbers from **start** to **stop-1** with a **step** size.

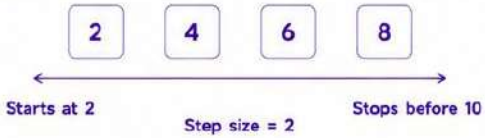
Syntax

```
range(start, stop, step)
```

Example

```
range(2, 10, 2)
```

Output



Useful when you want to **skip numbers** at regular intervals.

Key Takeaway: **range()** is **flexible** and helps us generate sequences in **many ways**.

Use the form that best fits your problem.

© Copyright Intellipaat. All rights reserved.

Understanding Step Size

The step value decides how numbers progress in the sequence.

```
range(1, 10, 3)
```

Output: 1, 4, 7



We start at 1, jump by 3 each time, and stop before 10.

1. Positive Step (Forward)

Counting forward.




```
range(1, 6, 1)
```

Output: 1, 2, 3, 4, 5

Default step is 1. It counts one by one.

2. Larger Positive Step (Skip Numbers)

Skipping numbers in between.



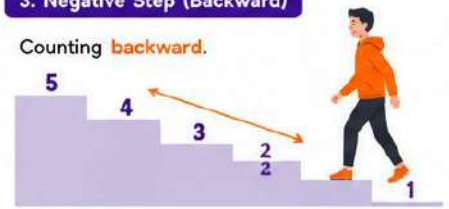

```
range(1, 10, 2)
```

Output: 1, 3, 5, 7, 9

Step = 2 means we skip one number each time.

3. Negative Step (Backward)

Counting backward.




```
range(5, 0, -1)
```

Output: 5, 4, 3, 2, 1

Negative step moves in the reverse direction.

Real-Life Analogy

Step = 1
You climb stairs one step at a time.

Step = 2 or 3
You take 2 or 3 steps at a time.

Negative Step
You come down one step at a time.

Key Takeaway

The step value gives you control over how the sequence progresses.

© Copyright Intellipaat. All rights reserved.

Why is range() Powerful

range() is **memory efficient** and ideal for loops.

Without range()

To print numbers from 1 to 1,000,000
we would need to create a huge list.

```
numbers = [1, 2, 3, 4, 5, ..., 1000000]
for i in numbers:
    print(i)
```



- ✗ Stores all numbers
- ✗ Uses more memory
- ✗ Hard to manage

With range()

We simply use **range()**.

```
for i in range(1, 1000001):
    print(i)
```



- ✓ Generates numbers when needed
- ✓ Uses less memory
- ✓ Fast and efficient

Real-Life Analogy



Writing all page
numbers in a notebook
(1, 2, 3, .. 1000000)

✗ Wastes space and effort



Counting page numbers
while turning pages
(1, 2, 3, ..)

✓ Smart and practical

Key Takeaway | **range()** creates sequences **efficiently** without storing everything at once.

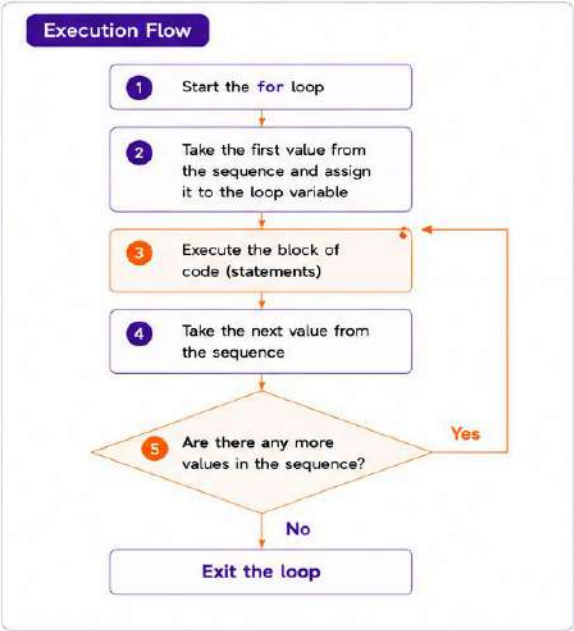
© Copyright Intellipaat. All rights reserved.

Working with sequence() in *for* Loop

© Copyright Intellipaat. All rights reserved.

How does it Work?

A for loop iterates over each item in a sequence and executes the block of code for that item



Example

```
for i in [10, 20, 30]:
    print(i)
```

Step by Step Iteration

Iteration	Value of i	Code Executed	Output
1	10	print(10)	10
2	20	print(20)	20
3	30	print(30)	30

Key Takeaway

The for loop automatically takes each item from the sequence, one by one, and executes the block of code until the sequence is completely exhausted.

© Copyright Intellipaat. All rights reserved.

Why does a for Loop need a sequence()?

Imagine This...



A for loop asks the same question!
It needs a collection of items to visit one by one.

Why Can't We Write?

```
for fruit:
    print(fruit)
```

Because Python doesn't know:

- Where should it start?
- What should it print?
- When should it stop?

A sequence answers all three questions.

Without a Sequence ✗

```
for item in ???:
    print(item)
```

Python doesn't know what to repeat over.

With a Sequence ✓

```
fruits = ["Apple", "Banana", "Mango"]

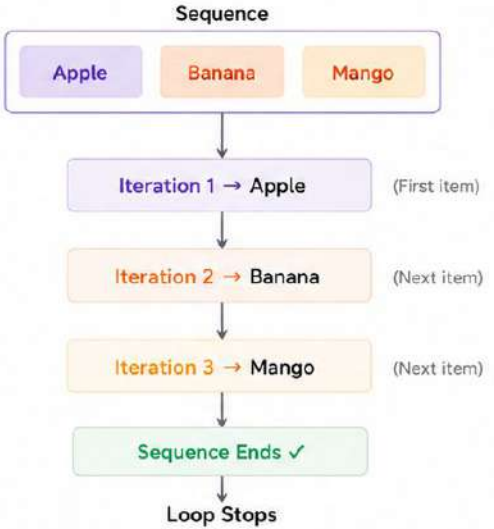
for fruit in fruits:
    print(fruit)
```

What Happens?

```
Iteration 1  →  Apple
Iteration 2  →  Banana
Iteration 3  →  Mango
```

The loop automatically picks one item at a time until there are no items left.

Visual Flow: How the for Loop Works



© Copyright Intellipaat. All rights reserved.

Syntax of using sequence() in for Loop

The for loop in Python is written using the following syntax.

Syntax

Keyword — Loop Variable — Sequence to iterate over (list, string, range, etc.)

```
for variable in sequence:
    statements
```

Block of code that will be executed in each iteration

Example

```
for i in [1, 2, 3]:
    print(i)
```

Output

```
1
2
3
```

Explanation

`for`	Starts the **for** loop.
`variable`	Takes each value from the **sequence** one by one.
`in`	Keyword that connects the variable with the **sequence**.
`sequence`	A collection of items (e.g., list, string, range) over which the loop iterates.
`statements`	Set of instructions that are executed in every iteration.

Key Idea

The **for** loop automatically handles the repetition for each item in the **sequence**.

© Copyright Intellipaat. All rights reserved.

Key Points About for Loop

Let's quickly recap the important concepts about **for** loop.

Remember

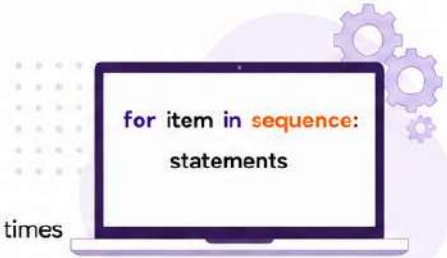
- **for** loop is used for **definite iteration**.
- Best when the number of repetitions is **known**.
- It iterates through a **sequence** of items.
- Python automatically handles moving to the next value.

Advantages of for Loop

1. Less code
2. Easy to read
3. Fewer mistakes
4. Cleaner logic

Example Applications

- Printing a series of numbers
- Traversing a list of items
- Processing student records
- Repeating tasks a fixed number of times



Key Takeaway

Use a **for** loop when you **know how many times** a task should repeat.

But where do sequences like **1, 2, 3, 4, 5** come from? Python provides a powerful built-in function called **range()**.

Let's learn how to generate such sequences using **range()**.

© Copyright Intellipaat. All rights reserved.

Hands-On: Generating Sequences with range()

© Copyright Intellipaat. All rights reserved.

Hands-On: Generating Sequences with range()

Problem Statement

A parking garage has reserved parking slots numbered **5, 10, 15, 20,** and **25.**

Use **range()** with a **start value**, **stop value**, and **step size** to generate these slot numbers. Print one message for every slot number generated.



Expected Output

```
Check Parking Slot: 5
Check Parking Slot: 10
Check Parking Slot: 15
Check Parking Slot: 20
Check Parking Slot: 25
```



© Copyright Intellipaat. All rights reserved.

Indefinite Looping: while Loop

© Copyright Intellipaat. All rights reserved.

Why Do We Need while Loop?

Sometimes, we **don't know** in advance how many times to repeat a task.

Real-Life Example



ATM PIN Entry

1. Enter PIN
2. If the PIN is **wrong**, it asks again.
3. If the PIN is **correct**, the process stops.

Question

Do we know how many attempts the user will need to enter the correct PIN?

No!

Therefore

We need a loop that runs until a **condition** changes.

while

The **while** loop is used when we **don't know** how many times the loop should run.

Start → Repeat while condition is **True** → Stop when condition is **False**

© Copyright Intellipaat. All rights reserved.

What is a while Loop?

A **while** loop executes a block of code as long as a given **condition** remains **True**.

Definition



A **while** loop repeatedly executes a set of statements as long as the **condition** is **True**.

Key Points

- It checks the condition **before** executing the block.
- If the condition is **True**, the block executes.
- If the condition becomes **False**, the loop stops.

Syntax

```
while condition:
    statements
```

This is evaluated first.

Code block
This runs repeatedly while condition is True.

Indentation is important!
The statements inside the **while** loop must be indented.

Key Takeaway | A **while** loop keeps running until the **condition** becomes **False**.

© Copyright Intellipaat. All rights reserved.

Anatomy of a Safe while Loop

Every reliable **while** loop has **three essential components**.

1 Initialization



Set the starting value of the control variable.

Example

```
count = 1
```

Starting point of the loop.

2 Condition



Check the condition. If **True**, the loop continues.

Example

```
count <= 5
```

Loop runs while condition is True.

3 Update



Change the value of the control variable.

Example

```
count += 1
```

Moves the loop towards stopping.

Why all three are important?

- Without **initialization**, we don't have a starting point.
- Without **condition**, we don't know when to stop.
- Without **update**, the loop may never stop (**infinite loop!**).

Key Takeaway

A safe **while** loop requires **Initialization**, **Condition**, and **Update**.

© Copyright Intellipaat. All rights reserved.

Example

Let's see a simple **while** loop in action.

Code

```
1  count = 1
2  while count <= 5:
3      print(count)
4      count += 1
```

Output

```
1
2
3
4
5
```

Step-by-Step Execution

Iteration	count (Before)	Condition (count <= 5)	Action	count (After)
1	1	1 <= 5 → True	print(1) count += 1	2
2	2	2 <= 5 → True	print(2) count += 1	3
3	3	3 <= 5 → True	print(3) count += 1	4
4	4	4 <= 5 → True	print(4) count += 1	5
5	5	5 <= 5 → True	print(5) count += 1	6
6	6	6 <= 5 → False	Loop stops	—

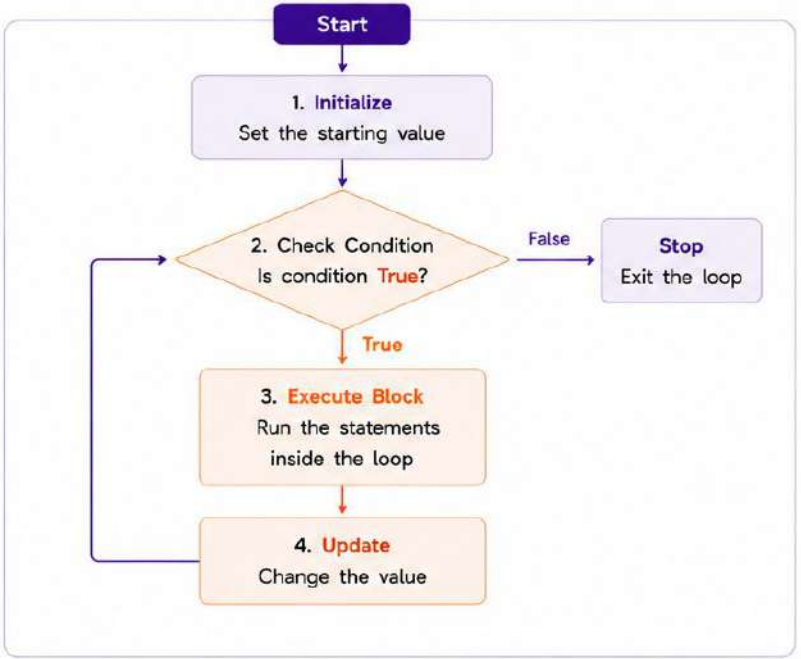
Key Takeaway

The **while** loop runs as long as the condition is **True**.
Here, it prints numbers from **1** to **5**.

© Copyright Intellipaat. All rights reserved.

How does a while Loop Work?

A **while** loop follows a simple cycle.



Explanation

1. **Initialize**
   Set the starting value of the control variable.
2. **Check Condition**
   If the condition is **True**, the loop continues.
   If **False**, the loop stops.
3. **Execute Block**
   The statements inside the loop are executed.
4. **Update**
   The value is updated to move towards the stopping condition.
   Then, the condition is checked again.

Key Takeaway

A **while** loop repeats steps **2** to **4** until the condition becomes **False**.

© Copyright Intellipaat. All rights reserved.

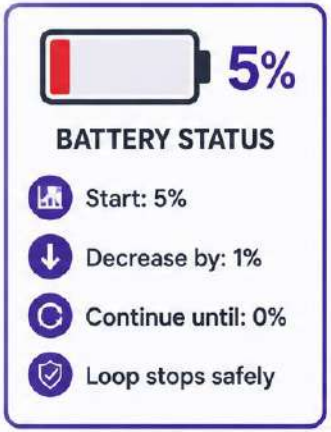
Hands-On: Indefinite Looping — while Loop

© Copyright Intellipaat. All rights reserved.

Hands-On: Indefinite Looping — while Loop

Problem Statement

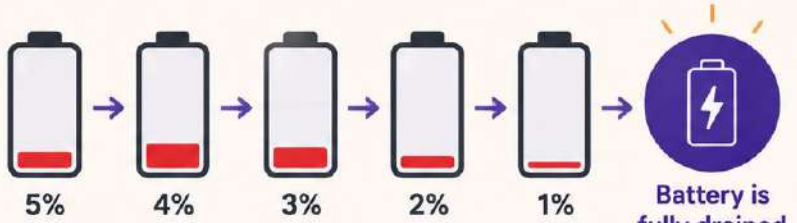
- A battery-powered device has 5% battery remaining.
- Use a `while` loop to display the current battery percentage, then reduce it by 1% during every iteration.
- Continue until the battery reaches 0%.
- Ensure that the loop variable is updated so that the loop stops safely.





Expected Output

```
Battery Level: 5%
Battery Level: 4%
Battery Level: 3%
Battery Level: 2%
Battery Level: 1%
Battery is fully drained.
```



© Copyright Intellipaat. All rights reserved.

Infinite Loops & Unsafe Looping

© Copyright Intellipaat. All rights reserved.

What is an Infinite Loop?

A loop that never reaches its stopping condition is called an infinite loop.

Look at the code on the right.

Will this loop stop?

Definition

An infinite loop is a loop that continues to execute forever because the stopping condition is never reached.

Example

```
1  count = 1
2  while count <= 5:
3      print(count)
```



No! This loop will never stop.

Why? Because **count** is never changed inside the loop.

So, **count** always remains 1, and the condition **count <= 5** is always **True**.

Key Takeaway

An infinite loop keeps running forever because the stopping condition is never reached.

© Copyright Intellipaat. All rights reserved.

Common Causes of Infinite Loops

Most infinite loops happen because of **simple mistakes**.

1 Missing Update

The loop variable is never updated inside the loop.

Example

```
1  x = 0
2  while x < 10:
3      print(x)
```

Why it's a problem?

x never changes, so the condition `x < 10` is always True.

2 Wrong Condition

The loop moves away from the stopping condition.

Example

```
1  x = 5
2  while x > 0:
3      print(x)
4      x += 1
```

Why it's a problem?

x keeps increasing (5 → 6 → 7 ...), so `x > 0` will never become False.

3 Condition Always True

The condition is always True (intentionally or unintentionally).

Example

```
1  while True:
2      print("Running")
```

Note

`while True` is sometimes used intentionally (e.g., menus, servers, game loops). Use with care!

Key Takeaway

Infinite loops usually occur due to **missing updates**, **wrong conditions**, or conditions that are **always True**.

© Copyright Intellipaat. All rights reserved.

How to Avoid Infinite Loops

Follow these simple guidelines to write safe and reliable while loops.

1 Update the Control Variable

Always change the loop variable inside the loop.

Example

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

count moves closer to 6, so the loop will stop.

2 Check the Stopping Condition

Ensure the condition can eventually become False.

Example

```
while count <= 5:
    ...
```

Will count eventually become 6?

Yes → Condition becomes False.

3 Dry-run Your Logic

Test 2–3 iterations on paper before running your code.

Example (Dry Run)

Iteration	count	Condition (count <= 5)
Start	1	True
1st pass	2	True
2nd pass	3	True
3rd pass	4	True
4th pass	5	True
5th pass	6	False

After count becomes 6, the condition is False and the loop stops.

Real-life Analogy



An infinite loop is like running on a treadmill without an OFF button — you keep running forever!

Always give your loop a way to stop.

Key Takeaway

A safe while loop has a clear plan:

Initialize → Check Condition → Execute → Update → Repeat

This ensures the loop will eventually stop.

© Copyright Intellipaat. All rights reserved.

Loop Interruption

© Copyright Intellipaat. All rights reserved.

Why Do We Need Loop Interruption?

Loops are great for repetition, but sometimes we need to **stop early** or **skip** certain iterations.

Scenario

We are checking a list of student marks.

45	67	82	−1	91
			↑ Invalid data	

Question:
What should we do when we encounter −1 (invalid data)?

Stop processing the rest of the data? or **Skip this value** and continue checking the remaining values?

The Solution

Python provides two special keywords that help us control the flow of a loop from inside:

break
Exit the loop immediately.

continue
Skip the current iteration and move to the next one.

Key Takeaway

Loop interruption helps us **make smarter decisions** while processing data.

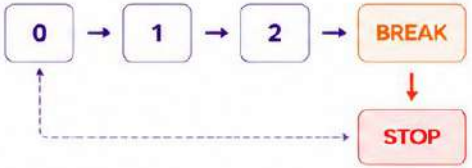
© Copyright Intellipaat. All rights reserved.

What is break?

The **break** statement immediately **terminates the loop**, even if the condition is still **True**.

1. What happens?

When **break** is executed,
the loop **stops immediately**.



At `x = 3`, **break** executes and the loop **terminates**.

2. Code Example

```
1  for x in range(5):
2      if x == 3:
4          break   # Exit the loop
5      print(x)
```

Walkthrough:

- x takes values `0`, `1`, `2` → printed
- When x becomes `3` → **break** executes
- Loop stops immediately, `4` is never checked

3. Output

Only values before **break** are printed.

```
0
1
2
```

Loop did not continue after `x = 2`.

Real-life Analogy



Fire alarm rings!
You leave the building immediately.
You don't wait for the next instruction.
That is exactly what **break** does.

Key Takeaway

Use **break** when we want to stop a loop early and **exit it completely**.

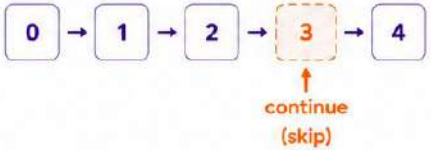
© Copyright Intellipaat. All rights reserved.

What is Continue?

The **continue** statement **skips the current iteration** and moves to the **next** one.

1. What happens?

When **continue** is executed,
the current iteration is **skipped**,
and the **loop continues**.



At x = 3, the current iteration is skipped, and control jumps to the next value.

2. Code Example

```
1  for x in range(5):
2      if x == 3:
4          continue  # Skip this iteration
5      print(x)
```

Walkthrough:

- x takes values 0, 1, 2 → printed
- When x becomes 3 → **continue** executes
- 3 is not printed, loop moves to next value (4)
- Loop completes normally

3. Output

All values are printed **except the skipped one**.

```
0
1
2
4
```

Loop did not stop.
Only the iteration with x = 3 was skipped.

Real-life Analogy



In an exam:
You don't know one question,
so you **skip** it and move to the next.
You come back later if time permits.
That is exactly what **continue** does.

Key Takeaway

Use **continue** when you want to **skip** the current iteration but **keep the loop running**.

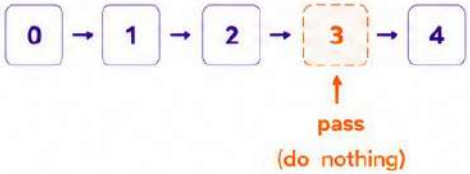
© Copyright Intellipaat. All rights reserved.

What is pass?

The pass statement does nothing. It is used as a placeholder.

1. What happens?

When pass is executed, nothing happens and the loop continues normally.



At x = 3, pass executes, does nothing, and the loop moves to the next value.

2. Code Example

```
1  for x in range(5):
2      if x == 3:
3          pass    # Do nothing
5      print(x)
```

Explanation:

- Loop starts and runs normally.
- When x becomes 3 → pass executes.
- Nothing happens, and loop continues.

3. Output

All values are printed.

```
0
1
2
3
4
```

pass is not used to control the loop. It simply keeps the flow unchanged.

Key Takeaway

Statement	Action
break	Stop the loop
continue	Skip the current iteration
pass	Do nothing

© Copyright Intellipaat. All rights reserved.

break vs continue vs pass

Three statements. Three **different actions** in a loop.

Statement	Action	Effect on Loop	Real-life Example
break	Immediately exits the loop.	Loop **stops** completely.	**Fire alarm rings** Leave the building immediately.
continue	Skips the current iteration.	Loop **continues** with the next iteration.	**Difficult question in an exam** Skip it and move to the next one.
pass	Does nothing.	Loop **continues** normally without any change.	**Leave a note: "To do later."** Do nothing for now.

At a Glance

break	Code runs	→	Condition met	→	STOP		Loop ends here.
continue	Code runs	→	Condition met	→	SKIP	→ NEXT ITERATION →	Loop continues.
pass	Code runs	→	Condition met	→	DO NOTHING	→ NEXT LINE →	Loop continues.

Key Takeaway

break stops the loop, **continue** skips the current iteration, and **pass** does nothing.

© Copyright Intellipaat. All rights reserved.

Hands-On: Loop Interruption

© Copyright Intellipaat. All rights reserved.

Hands-On: Loop Interruption

Problem Statement

Create three short loop demonstrations:

1. Use **break** to stop checking seat numbers when **Seat 4** is reached.



2. Use **continue** to skip **Seat 4** while displaying the remaining seat numbers from 1 to 6.



3. Use **pass** as a placeholder when the loop reaches **Checkpoint 3**; the loop must continue normally because **pass** does not stop or skip an iteration.



Expected Output

Break Demo:

```
Checking Seat: 1
Checking Seat: 2
Checking Seat: 3
Seat 4 reached. Stopping check.
```

Continue Demo:

```
Available Seat: 1
Available Seat: 2
Available Seat: 3
Available Seat: 5
Available Seat: 6
```



Pass Demo:

```
Checkpoint 1 completed.
Checkpoint 2 completed.
Checkpoint 3 completed.
Checkpoint 4 completed.
```

© Copyright Intellipaat. All rights reserved.

State Tracing Strategies

© Copyright Intellipaat. All rights reserved.

Why Do We Need State Tracing?

Loops repeat many times. Variables **change** at each step.
It becomes hard to predict the final result without **tracking the changes**.

Example

Consider the following code:

```
1  total = 0
2  for x in range(1, 5):
3      total += x
4      print(x, total)
5  print("Final total:", total)
```

Question:
What will be the final value of **total**?

The Challenge

Without tracing, it's easy to get confused:

- How does **total** change after each iteration?
- What value does **x** take at each step?
- Why is the final answer what it is?

To understand loops clearly, we must **track variable values** after each iteration.

This is where **State Tracing** comes in!

© Copyright Intellipaat. All rights reserved.

What is State Tracing?

State tracing means **recording how variables change** during program execution, especially inside loops.

1. What is State Tracing?



It is the process of **writing down the values** of important variables **after each iteration** of a loop.



2. What Do We Track?

During state tracing, we focus on the following:

Loop Variable
The value of the loop counter (like `i`, `x`, `n`) that changes each iteration.

Accumulator / Variable
Values that store or update results (like **sum**, **total**, **count**).

Condition Values
The result of the conditions being checked.

Output (if any)
What gets printed or returned.

3. Why Does It Help?

Understand Loop Execution
Helps us see how values change at each step.

Debug Errors Easily
Makes it easier to find where logic goes wrong.

Predict Final Output
Allows us to confidently determine the final result.

Key Takeaway

State tracing helps us understand loops **step by step** by tracking how variables change.

© Copyright Intellipaat. All rights reserved.

Tracing Table Example

Let's trace the values of variables **step by step.**

Example Code

```
1  total = 0
2  for x in range(1, 4):
3      total += x
4      print(x, total)
5  print("Final total:", total)
```

What Happens?

- **total** starts at 0.
- In each iteration, **x** is added to **total**.
- We record the values **after every iteration.**

Trace Table

Iteration	x (Loop Variable) range(1, 4)	total (Accumulator) total += x	What Happens in This Step?
Start (Before loop)	—	0	**total** is initialized to 0.
1st Iteration	1	1 (0 + 1)	**x** = 1 **total** = 0 + 1 = 1
2nd Iteration	2	3 (1 + 2)	**x** = 2 **total** = 1 + 2 = 3
3rd Iteration	3	6 (3 + 3)	**x** = 3 **total** = 3 + 3 = 6
After Loop (Final State)	—	6	Loop ends. Final value of **total** is 6.

Output

```
1 1
2 3
3 6
Final total: 6
```

← Printed by the program

The tracing table helps us see how **total** grows in each iteration.

Key Takeaway

A tracing table turns the invisible steps of a loop into a **clear, step-by-step** picture.

© Copyright Intellipaat. All rights reserved.

Best Practices for Tracing

Follow these steps whenever you want to trace a loop.



1 Identify Initial Values
What are the starting values of variables before the loop begins?



2 Identify What Changes
Which variables change in each iteration and how?



3 Check the Condition
What is the loop condition?
When will it become False?



4 Record Each Step
Make a table and record the values after every iteration.



5 Determine the Final Result
What are the final values and what will be printed/returned?

Tips for Effective Tracing

- Trace with small inputs first.
- Focus on one variable at a time.
- Write neatly and clearly.
- Don't guess — trace!

Why It Matters?

- Builds accuracy in thinking.
- Helps find logical errors early.
- Strengthens problem-solving skills.

Key Takeaway

If a loop feels confusing, don't guess — trace it step by step!

© Copyright Intellipaat. All rights reserved.

Hands-On: State Tracing Strategies

© Copyright Intellipaat. All rights reserved.

Hands-On: State Tracing Strategies

Problem Statement

- A delivery vehicle starts the day with **50 packages**.
- During three delivery stops, it delivers **8**, **12**, and **15** packages respectively.
- Use a **loop** to subtract the delivered packages from the total.
- Create a **tracing table** that shows the stop number, the packages delivered, and the remaining packages after every iteration.



Expected Output

Stop	Packages Delivered	Remaining Packages
1	8	42
2	12	30
3	15	15

Packages Remaining: 15



© Copyright Intellipaat. All rights reserved.

Thanks!


080-4524-9465


support@intellipaat.com

© Copyright Intellipaat. All rights reserved.